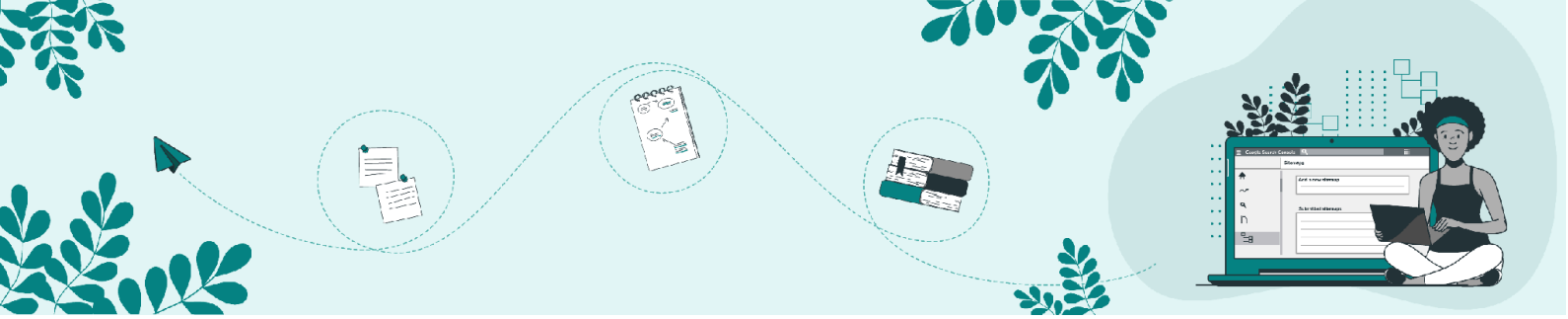


Caro, estudante.

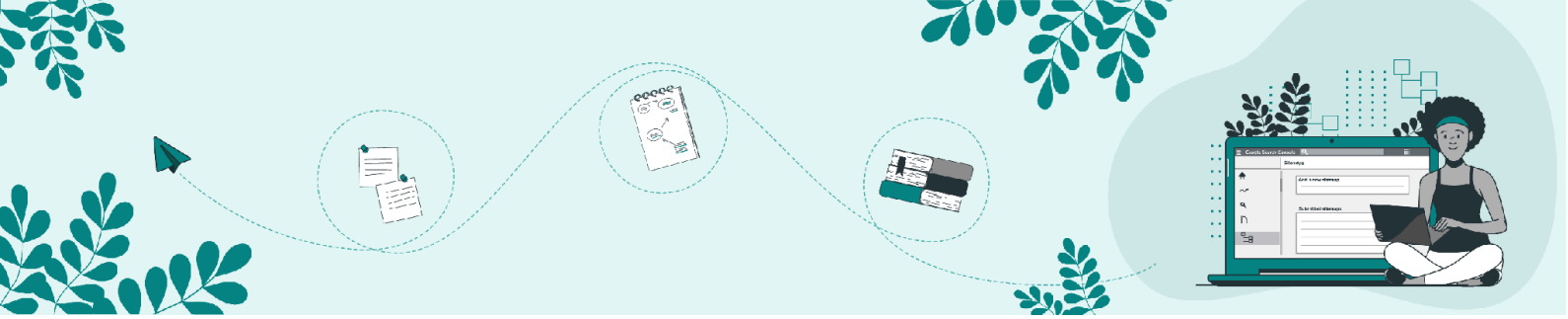
Agora que você se apropriou dos conteúdos abordados e das situações-problema nesta disciplina, chegou o momento de testar seus conhecimentos.
Escolha uma das 3 situações-problema que você leu no material e proponha um projeto de intervenção. Você deve descrever:

- **Objetivo:** como você pretende solucionar a situação-problema escolhida;
- **Revisão de Conceito:** conhecimentos adquiridos no curso utilizados como base de estudo;
- **Metodologia:** qual abordagem, técnica ou processo usados para resolver o problema indicado;
- **Tempo:** quanto tempo gastou para a solução;
- **Procedimento:** indique o passo-a-passo para a resolução, e possíveis materiais utilizados;
- **Resultado:** o que resultou o processo.

Nome: Alexandro Castro
RGM: 35041536

Qual situação-problema você escolheu para criar o seu projeto de intervenção?
☒ Situação-problema 1
☐ Situação-problema 2
☐ Situação-problema 3

A Situação-Problema escolhida:
Sobre o que é a situação-problema escolhida por você? Descreva em até 5 linhas.
A situação-problema 1 é sobre SQL Injection. O risco aparece quando eu concateno o que a pessoa digitou direto na query, e esse texto vira parte do comando SQL. Com isso alguém consegue ler, alterar ou apagar dado que não devia. Fala também de como a modelagem (tipos, constraints, índices) afeta segurança e eficiência.



Objetivo:

Fazer o Catálogo de Livros, Séries e Filmes (Java, Servlets, JSP e PostgreSQL) sem concatenar string em SQL, usando PreparedStatement no padrão DAO, e provar com teste que a busca não deixa uma entrada maliciosa virar comando.

Revisão de Conceito:

Usei JDBC com PreparedStatement (parâmetros com ?), o padrão DAO para deixar todo SQL num lugar só, Servlets e JSP com JSTL/EL (MVC) e validação de formulário. Na modelagem, tipos certos, NOT NULL, CHECK e índices. Também tratamento de exceção com try-with-resources e c:out para escapar a saída.

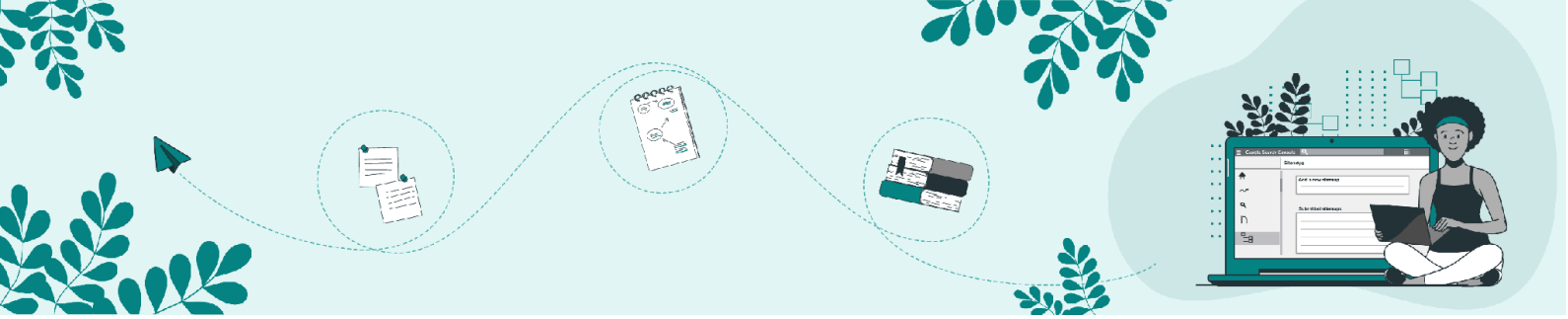
Metodologia:

Fui de DAO + PreparedStatement para blindar o SQL, e Servlet como controlador com JSP só para exibir (MVC). Para validar, escrevi testes JUnit que mandam texto de ataque na busca, como '; DROP TABLE item, e conferem que volta zero resultado e que a tabela continua lá.

Tempo:

Leitura do material e escolha da situação: 1h.
Modelagem e script SQL: 1h.
Item, conexão e DAO: 1h30. Servlet e JSPs: 2h.
Testes(com tentativa de injection): 1h.
Preencher e revisar esta atividade: 30min.
Total: 7h.

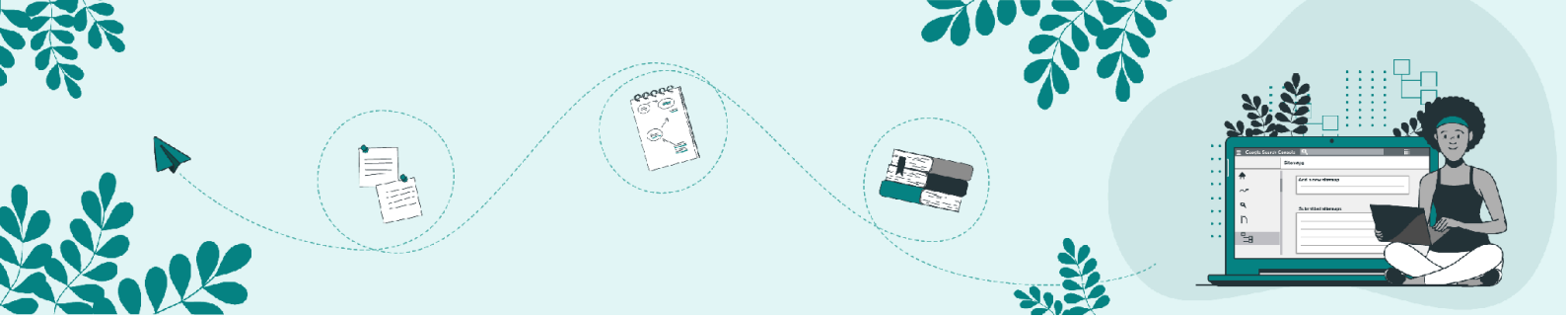
Procedimento e material utilizado:



1) Li as 3 situações e escolhi a
1. 2) Modelei a tabela item no PostgreSQL com NOT NULL, CHECK (ano, tipo, nota) e índice em lower(titulo) e lower(autor), e guardei no schema.sql.
3) Criei o projeto Maven (WAR) com Servlet API, JSTL e driver do PostgreSQL.
4) Fiz a classe Item e o ItemDAO, com PreparedStatement em todo comando (inserir, atualizar, excluir, buscar, listar).
5) Fiz o ItemServlet (listar, ver, novo, editar, salvar,excluir) e as telas JSP.
6) Validei o formulário no Item.validar() e tratei SQLException no servlet: loga o erro real e mostra mensagem genérica.
7) Escrevi testes JUnit, incluindo a busca com ' OR '1'='1 e '; DROP TABLE item; --. 8) Subi com Jetty e testei no navegador e com curl. Ferramentas: Java, Maven, PostgreSQL, JUnit 5.

Resultado e discussão:

Qual foi a resolução da situação-problema após a aplicação do procedimento? Faça também comentários finais sobre o projeto. Descreva em até 15 linhas.
O catálogo ficou funcionando: cadastrar, listar, ver detalhes, editar, excluir e buscar por título ou autor/diretor, mais os extras de filtro por gênero, nota em estrelas e link de capa. A busca recebeu '; DROP TABLE item; -- e ' OR '1'='1 e voltou zero itens, com a tabela intacta, porque o PreparedStatement manda o texto como dado e não como SQL. Se eu tivesse concatenado, essa mesma entrada poderia mudar a lógica da consulta. No formulário, dado inválido volta com mensagem e nem chega no banco, e o banco ainda tem os CHECKs como segunda barreira. Na minha opinião a maior lição é que segurança não é um passo no final, é a forma de escrever a query desde a primeira linha. Dá pra melhorar: autenticação,



proteção CSRF nos POSTs, pool de conexões (hoje abro uma por consulta) e uma camada de serviço. Mas pro escopo da atividade, atendeu.